

Getting started

You will need to install the following packages: *csound* and *csound-python*.

Csound requires an XML-like configuration file with three sections. Create a minimal file, *tutorial.csd*:

```
<CsoundSynthesizer>
<CsOptions>
</CsOptions>
<CsInstruments>
</CsInstruments>
<CsScore>
</CsScore>
</CsoundSynthesizer>
```

You will be satisfied with the default options. The instruments section is where we define the 'orchestra', which comprises all the instruments that will be used to create our musical masterpiece. These are the same as the contents of *myfile.orc* in the beginner's articles in www.csounds.com/journal/articleIndex.html.

The score section will contain a list of instructions for the instruments in the orchestra. These are the same as the contents of *myfile.sco*, mentioned in the articles mentioned earlier.

Create the same simple instrument as in the beginners' introduction, www.csounds.com/ezine/winter1999/beginner/index.html:

```
<CsInstruments>
sr=44100 ; Sample Rate
kr=22050 ; Control Rate
ksmps=2 ; sr/kr As far as I know this is always the case
nchnls=2 ; 1=mono, 2=stereo, 4=quad
instr 1 ; Instrument 1 begins here
aout oscil 10000, 440, 1 ; An oscillator
outs aout, aout ; Output the results to a stereo sound file
endin ; Instrument 1 ends here
</CsInstruments>
```

The first four lines have header information, which controls the output format. The remaining lines are the definition of the instrument, which is a simple oscillator operating at a frequency of 440Hz and a volume of 10000, about a third of the maximum. (Volume is represented as a 16-bit integer.) The third parameter to the *oscil* command/opcode identifies the waveform table to be used in the score given below. The oscillator is given a variable name *aout*. The same sound is passed to both the channels.

Now, add the score:

```
<CsScore>
f 1 0 16384 10 1 : table #, start time, the size, generator, parameter
i 1 0 1 : instrument #, start time, duration
</CsScore>
```

f is the waveform table, which is available at the start of performance with 16,384 samples. The generator value 10 with parameter 1 corresponds to a sine wave in Csound.

The *i* line is an instrument event with the instrument number, the start time and the duration in seconds as the parameters.

You can run the following script, get a wave file and play it:

```
$ csound -Wo tutorial.wav tutorial.cs
$ aplay tutorial.wav
```

Controlling Csound using Python

Write the following code in *tutorial.py*. This is pretty useless as all it does is replace the command line for executing Csound. But have patience.

```
# Import the Csound API extension module.
import csnd

# Create an instance of Csound.
csound = csnd.CppSound()

# Enable Csound to print console messages
csound.setPythonMessageCallback()

# Load the tutorial piece created earlier.
csound.load('tutorial.csd')

# Set the Csound command for off-line rendering.
csound.setCommand('csound -Wo tutorial.wav temp.orc temp.sco')

# Export the .orc and .sco file for performance
csound.exportForPerformance()

# Actually run the performance.
csound.perform()
```

We will now add code to generate the score algorithmically. However, our current instrument is pretty hopeless. So, you will need to refine the instrument in *tutorial.csd* as follows:

```
instr 1
iamp = p4
ifqc = p5
itabl1 = p6
aout oscil iamp, ifqc, itabl1
outs aout, aout
endin
```

The new oscillator uses the amplitude, frequency and waveform table reference as parameters. The variables in Csound have a strict format reminiscent of Fortran. Local variables start with the letters 'i', 'k' or 'a'. A variable